

HW8 Step III

Database Comparison Report

Nithish Reddy Vemula | CS 6650 Distributed Systems | March 22, 2026

Purpose. This report compares the MySQL and DynamoDB shopping cart implementations using only the measured results in combined_results.json and the Step I and Step II load testing summaries. Every recommendation below is tied back to those measurements.

1. Data Verification

Verification result. Both raw result files contain exactly 150 operations, with 50 create_cart, 50 add_items, and 50 get_cart operations each. Both datasets also achieved 150 out of 150 successful operations, so the comparison is valid and traceable to the merged file.

Backend	Total Ops	Create	Add	Get
MySQL	150	50	50	50
DynamoDB	150	50	50	50

Source of truth used for this report: combined_results.json.

2. Performance Comparison

Table 1 summarizes the direct 150 operation sequential test results.

Metric	MySQL	DynamoDB	Winner	Margin
Avg Response Time (ms)	98.55	107.48	MySQL	8.93 ms
P50 Response Time (ms)	123.17	128.34	MySQL	5.17 ms
P95 Response Time (ms)	132.06	139.53	MySQL	7.47 ms
P99 Response Time (ms)	142.11	191.89	MySQL	49.78 ms
Success Rate (%)	100%	100%	Tie	0
Total Operations	150	150		

MySQL is faster on every latency metric in the sequential run, while both systems achieved the same success rate. The biggest separation is at P99, where DynamoDB shows noticeably higher tail latency.

Table 2 breaks the averages down by operation type.

Operation	MySQL Avg (ms)	DynamoDB Avg (ms)	Faster By
CREATE_CART	42.93	55.43	MySQL by 12.50 ms
ADD_ITEMS	127.86	133.38	MySQL by 5.51 ms
GET_CART	124.87	133.65	MySQL by 8.79 ms

Interpretation. The create_cart gap is the largest because MySQL performs a single INSERT with AUTO_INCREMENT, while DynamoDB needs an atomic counter update plus the cart write. The add_items gap is smaller because both designs need multiple steps to modify cart contents.

3. Concurrent Load Behavior

The Step I and Step II Locust summaries reveal the more important architectural difference: scaling behavior under concurrency.

Metric	MySQL 10u	DynamoDB 10u	MySQL 50u	DynamoDB 50u	MySQL 100u	DynamoDB 100u
RPS	30.7	30.6	152.4	149.9	302.8	133.3
Avg (ms)	21.44	26.35	22.98	26.95	45.98	370.72
P95 (ms)	25	31	26	32	31	840
Max (ms)	239	131	634	256	7031	1616
Failures	0%	0%	0%	0%	0%	0%

Evidence based finding. At 10 and 50 users, both systems are close. At 100 users, MySQL scales to 302.8 RPS while DynamoDB drops to 133.3 RPS. This indicates that the DynamoDB design is limited more by the application level read modify write pattern than by raw service capacity.

4. Consistency Model Impact

Observed behavior. Across 50 consistency checks reported in Step II, no stale reads were observed with DynamoDB. In this assignment, eventual consistency did not create a visible user experience issue during normal cart reads.

Important caveat. The bigger risk is not stale reads but lost updates under concurrent read modify write cycles. MySQL protects multi step cart updates with transactions and database managed atomicity, while the current DynamoDB implementation would need UpdateItem based atomic updates or optimistic locking to prevent overwrite races.

5. Database Recommendation Framework

Startup MVP, low traffic, 1 developer, minimal ops burden: DynamoDB. DynamoDB is a good fit when operational simplicity matters most. The low concurrency tests show similar performance to MySQL, and the fully managed model removes connection pool tuning, patching, and instance sizing overhead.

Growing product with richer reporting and transactional workflows: MySQL. MySQL is the stronger default when the application needs joins, relational integrity, and predictable transactional behavior. The tests also show materially better throughput at 100 users.

High concurrency cart updates with current implementation unchanged: MySQL. The measured data favors MySQL because its add item path stays efficient under load, while the DynamoDB read modify write design becomes the bottleneck.

Simple key based workload with bursty traffic and limited maintenance capacity: DynamoDB. If the access pattern stays simple and item level atomic operations are used, DynamoDB remains attractive because it avoids infrastructure management and can scale without manual provisioning.

6. Reflection and Final Conclusion

This assignment shows that database selection is not just about SQL versus NoSQL syntax. It depends on access patterns, contention behavior, and how much application logic is pushed into the database layer.

Based on the actual test data, MySQL is the better choice for this shopping cart implementation as written because it delivered lower sequential latency, much higher throughput at 100 users, and stronger transactional safety for concurrent cart modification.

DynamoDB is still a valid recommendation for simpler or more operationally constrained systems, but the evidence here says it would need a better write pattern before it becomes the preferred option for this workload.

Appendix: Traceability

Sequential latency metrics and operation counts were taken from combined_results.json. Concurrent load metrics and consistency observations were taken from the Step I and Step II reports. The attached verification screenshot shows the merged file counts used as the foundation for this report.